UNIT-IV
POINTERS & USER DEFINED TYPES

Introduction:

- Pointer is one of the strengths of 'C' language.
- A pointer is a powerful technique to access the data by indirect reference as it holds the address of the variable where it has been stored in the memory.

POINTER DEFINITION

A **pointer** is a variable whose value is the address of another variable, i.e., direct address of the memory location. Like any variable or constant, it is required to declare a pointer before using it to store any variable address. The general form of a pointer variable declaration is:

type *var-name;

Here, **type** is the pointer's base type i.e., the type of variable whose address it holds; it must be a valid C data type and **var-name** is the name of the pointer variable. The asterisk * is used to designate a variable as a pointer. Following are the valid pointer declaration:

```
int    *ip;    /* pointer to an integer */
double *dp;    /* pointer to a double */
float  *fp;    /* pointer to a float */
char   *ch     /* pointer to a character */
```

Advantages:

The pointer has the following advantages.

- It allows to pass variables, arrays, functions, string, and structures as function arguments.
- A pointer allows to return structured array variables from functions.
- It provides functions that can modify their calling arguments.
- It supports dynamic allocation and free allocation of memory segments.
- Only with a pointer, complex data types can be declared and accessed easily.
- With the help of a pointer, variables can be swapped without physically moving them.
- The pointers are more efficient in handling data tables.
- The pointers reduce the length and complexity of a program.

POINTER OPERATORS

There are two special operators that are used with pointers. They are -

1) Address Operator (&) 2) Dereferencing (Pointer) operator (*)

1) Address Operator (&) - The Address Operator (&) is a unary operator that returns the memory address of its operand, for example **bal=&balance;**
puts into **bal** the memory address of the variable **balance**. This address is the location of the variable in the computer's internal memory. It has nothing to do with the value of balance. The statements can be verbalized as **bal** receives the address of **balance**.

2) **Dereferencing (Pointer) operator (*)** - The second operator is Dereferencing (Pointer) operator (*), which is also a unary operator that returns the value of the variable at the address specified by its operand. Consider the example: **data=*balance;**
This operation will assign the value stored at memory address referenced by **balance** into the variable **data**.

Example Program to show the use of Pointer Operators

```
#include <stdio.h>
void main ()
{
  int  var = 20;   /* actual variable declaration */
  int  *ip;        /* pointer variable declaration */
  ip = &var;  /* store address of var in pointer variable*/
  printf("Address of var variable: %x\n", &var  );
  printf("Address stored in ip variable: %x\n", ip );
  printf("Value of *ip variable: %d\n", *ip );
}
```

When the above code is compiled and executed, it produces result something as follows:

Address of **var** variable: bffd8b3c
Address stored in **ip** variable: bffd8b3c
Value of ***ip** variable: 20

POINTER ARITHMETIC

Pointer Arithmetic is the set of valid arithmetic operations that can be performed on pointers. There are only a few operations that are allowed to perform on Pointers in C language. These operations are:

- Increment/Decrement of a Pointer
- Addition of integer to a pointer
- Subtraction of integer to a pointer
- Subtracting two pointers of the same type

Increment/Decrement of a Pointer - When a pointer is incremented, it actually increments by the number equal to the size of the data type for which it is a pointer. To understand what occurs, let **p1** be an integer pointer with a current value of 2000. Also, assume **ints** are 2 bytes long. After the expression → p1++;
p1 contains 2002, not 2001. The reason for this is that each time **p1** is incremented, it will point to the next integer. The same is true for decrements. For example, if **p1** has the value 2000, the expression → p1--;
causes **p1** to have the value 1998.

Addition of integer to a Pointer - When a pointer is added with an integer value, the value is first multiplied by the size of the data type and then added to the pointer. For example, if **ptr** is an integer pointer that stores **1000** as an address. If integer 5 is added to it using the expression, **ptr = ptr + 5**, then, the final address stored in the **ptr** will be ptr = 1000 + sizeof(int) * 5 = 1010.

Subtraction of integer to Pointer - When a pointer is subtracted with an integer value, the value is first multiplied by the size of the data type and then subtracted from the pointer.
For Example, For example, if **ptr** is an integer pointer that stores **1000** as an address. If integer 5 is added to it using the expression, **ptr = ptr - 5**, then, the final address stored in the **ptr** will be ptr = 1000 - sizeof(int) * 5 =990.

Subtraction of two Pointers - The subtraction of two pointers is possible only when they have the same data typed addresses. The result is generated by calculating the difference between the addresses of the two pointers and calculating how many bits of data it is according to the pointer data type.
For Example, if two integer pointers say **ptr1(address:1000)** and **ptr2(address:1002)** are subtracted. The difference between addresses i.e., **ptr1-ptr2** is **2 bytes**.

Comparison of Pointers – Comparison of pointers using any relational operator is allowed in C language. The result of such comparison is TRUE(1) for a satisfied condition or FALSE(0) for an unsatisfied condition.

Example program showing Pointer Arithmetic

```
void main()
{
   int x[]={24,17,14,35,12,29},*p1,*p2;
   p1=x;
   p2=&x[3];
   printf("Using Pointes\n");
   //using increment & decrement
   p1++;
   printf("Value at x[1]=%d\n",*p1);
   p2--;
   printf("Value at x[2]=%d\n",*p2);
   //using addition and subtraction
   p1=p1+2;
   printf("Value at x[3]=%d\n",*p1);
   p2=p2-2;
   printf("Value at x[0]=%d\n",*p2);
   //Subtraction of pointers
   printf("Ptr1-Ptr2 is %d bytes\n",p1-p2);
}
```

Output

```
Using Pointes
Value at x[1]=17
Value at x[2]=14
Value at x[3]=35
Value at x[0]=24
Ptr1-Ptr2 is 3 bytes
```

MANIPULATION OF ARRAYS USING POINTERS

Pointers and arrays are closely related but are not the same. In C language, the name of an array is actually a pointer to the first element of the array. That means, the memory address of the first element is the same as the name of the array. For example, an array is declared as → **int data[5];**
Then, the elements of the array are accessed internally using the pointer notation. That is.,
data[0] is equivalent to ***(data + 0)** and **&data[0]** is equivalent to **data + 0**

data[1] is equivalent to ***(data + 1)** and **&data[1]** is equivalent to **data + 1**
data[2] is equivalent to ***(data + 2)** and **&data[2]** is equivalent to **data + 2**
……
……
data[i] is equivalent to ***(data + i)** and **&data[i]** is equivalent to **data + i**

An array name is a permanent pointer to the base address or first element of the array and is unmodifiable. Hence, as shown in the above example the content of **data** (name of the array) is accessible but not modifiable.

As the name of the array is a permanent pointer to first element of the array, C language **allows manipulation of arrays using pointers**. The below example shows this.

```
void main()
{
   int arr[]={24,17,14,35,12,29},*ptr,i;
   ptr=arr;
   printf("Printing array elements using pointers...\n");
   for(i=0;i<6;i++)
   {
      printf("%d\t",*ptr);
      ptr++;
   }
}
```

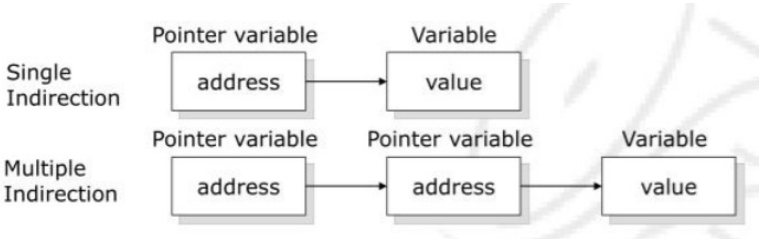
Output

Printing array elements using pointers...

24 17 14 35 12 29

As shown in the above example, the statement **ptr=arr;** works because **arr** is a pointer. Another way to say exactly the same thing would be to replace **ptr=arr;** with **ptr=&arr[0];**. Since **arr** contains the address of **arr[0]**, **arr** and **&arr[0]** mean the same thing.
The **arr** is a permanent pointer and cannot be changed, but **ptr** is not subject to such restrictions. Hence the pointer increment, **ptr++**, increments **ptr** the appropriate number of bytes to move it to the next element of the array.

Multiple Indirection (pointer to pointer)
The pointer to a pointer is a form of multiple indirections (or) a class of pointers. In case of a pointer to a pointer, the first pointer contains the address of second pointer which points to variable that contains values desired.
Multiple indirections can be carried on to whatever extent desired. But these are a few cases where more pointer to pointer is needed. Excess indirection is difficult to follow and process as it leads to conceptual error.



A variable that is pointer to a pointer must be declared by placing an additional asterisk(*****) in front of the variable name.**Ex :** int **ptr; Where **ptr** is a pointer which holds the address of another pointer.
To access the target value indirectly pointed to by a pointer to a pointer, it requires to apply the asterisk operator twice.

Void Pointers
When a pointer is declared as type void it is called void pointer. It is a generic pointer that cannot be dereferenced. Void pointers are used to point to data of different types. It is required to cast the type of the generic pointer using type casting operator { **(type*)** } to access the value of that type it points to.

The following code illustrates Generic Pointers.

```
void main()
{
   int i=10;
   char ch='A';
   float f=25.75;
   void *p;
   p=&i;
   printf("Using Void Pointer\n");
   printf("Value of integer i=%d\n",*(int*)p);
   p=&ch;
   printf("Value of character ch=%c\n",*(char*)p);
```

Output

Using Void Pointer

Value of integer i=10

Value of character ch=A

Value of float f=25.750000

```
    p=&f;
   printf("Value of float f=%f\n",*(float*)p);
}
```

Here in first print statement, generic is prefixed by ***(int*)**. This is called type casting in C language. Type is used to caste a variable from one data type to another datatype to make it compatible to the lvalue.

DYNAMIC ALLOCATION FUNCTIONS

In 'C' memory can be allocated in two ways

a. Static memory allocation
b. Dynamic memory allocation

Static Memory Allocation

The memory that is allocated during the compile time of a program. In this case once the size is declared it can't be altered i.e., it can't be increased or decreased. In static memory there are the following two disadvantages.

- Memory overflow or wastage of memory
- Memory under flow or in sufficient of memory

Dynamic Memory Allocation

The process of allocating memory at runtime is known as Dynamic memory allocation. There are four library functions known as "Dynamic memory management functions" that can be used for allocating and freeing memory during program execution i.e., runtime.

malloc()

The name malloc stands for "memory allocation". The function malloc() reserves a block of memory of specified size and return a pointer of type void which can be casted into pointer of any form.

Syntax

ptr=(cast-type*)malloc(byte-size)

Here, *ptr* is pointer of cast-type. The malloc() function returns a pointer to an area of memory with size of byte size. If the space is insufficient, allocation fails and returns NULL pointer.

Ex:- ptr=(int*)malloc(100*sizeof(int));

This statement will allocate 200 bytes of memory and returns a pointer points to the address of first byte of memory.

calloc()

The name calloc stands for "contiguous allocation". The only difference between malloc() and calloc() is that, malloc() allocates single block of memory whereas calloc() allocates multiple blocks of memory each of same size and sets all bytes to zero.

Syntax ptr=(cast-type*)calloc(n,element-size);

This statement will allocate contiguous space in memory for an array of *n* elements. For example:

ptr=(float*)calloc(25,sizeof(float));

This statement allocates contiguous space in memory for 25 elements each of size of float, i.e, 4 bytes.

free()

Dynamically allocated memory with either calloc() or malloc() does not get return on its own. The programmer must use free() explicitly to release space.

Syntax free(ptr);

This statement cause the space in memory pointer by ptr to be deallocated.

realloc()

If the previously allocated memory is insufficient or more than sufficient. Then, the realloc() is used to change memory size previously allocated using malloc() or calloc().

Syntax : ptr=realloc(ptr,newsize);

STRUCTURES

A structure in C is a user-defined data type that can be used to group variables or items of possibly different types. The items in the structure are called its member and they can be of any valid data type. Member variables are related to each other, and they are combined to provide complete information about the structure.

Structure Declaration

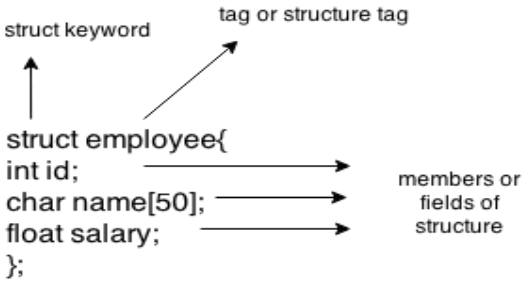
A Structure can be declared by using the **struct** keyword and specifying all the members of it along with their datatypes.

Syntax

```
struct structure_name
{
   data_type member1;
   data_type member2;
   .
   .
   data_type memeberN;
};
```

Example

```
struct employee
{
   int id;
   char name[50];
   float salary;
};
```



The above syntax is also called a structure template or structure prototype and no memory is allocated to the structure in the declaration.

Structure Definition

To use a structure in a program, it is required to declare a variable of declared structure type. This is known as structure definition. A structure variable can be declared in two ways:

1. Structure Variable Declaration with Structure Template

```
struct structure_name {
   data_type member_name1;
   data_type member_name1;
   ....
   ....
}variable1, varaible2, ...;
```

Example

```
struct employee
{
   int id;
   char name[50];
   float salary;
}e1,e2;
```

2. Structure Variable Declaration after Structure Template

```
// structure declared beforehand
struct structure_name variable1, variable2, .......;
```

Example

```
struct employee
{
   int id;
   char name[50];
   float salary;
};

struct employee e1,e2;
```

Accessing Structure Members

Structure members in a structure can be accessed by using member selection operator or period(Dot) operator (.).

Syntax StructureVaraiableName.MemberName;

Examples

```
e1.id = 10
e1.salary = 1052.75
scanf("%d",&e1.id);
strcpy(e1.name, "Narayana");
```

Structure Initialization

Assigning values to the structure members is known as structure initialization. A structure can be initialized by using an initializer list while defining a variable for a declared structure.

Syntax struct StructureName str = { value1, value2, value3 };

In this initialization, the values are assigned in sequential order as they are declared in the structure template.

Ex: struct employee e1 = { 101, "Narayana", 10234.75 };

Operations on Structures

In C language, the only operation that can be applied to **struct variables** is assignment. That means, if S1 and S2 are two structure variables defined of same structure type then statement **S1= S2** is valid and assigns S2 to S1 i.e., the member values of S1 are copied to the corresponding member values of S2.

Example program on Structures

```
struct student
{
   int id;
   char gd;
   float avg;
}s1;
void main()
{
   struct student s2, t = { 101, 'F', 87.5 };
   printf("Enter ID, Gender and Average:");
   scanf("%d %c %f",&s2.id, &s2.gd, &s2.avg);
   printf("Student2::%d %c %f\n",s2.id,s2.gd,s2.avg);
   s1=t;
   printf("Student1::%d %c %f",s1.id,s1.gd,s1.avg);
}
```

Output

```
Enter ID, Gender and Average: 102 M 85.25

Student2::102 M 85.250000

Student1::101 F 87.500000
```

Arrays within Structures

A structure may contain elements of different data types – int, char, float, double, etc. It may also contain an array as its member. Such an array is called an array within a structure. An array within a structure is a member of the structure and can be accessed using dot operator.

Example

```
struct employee
{
   int id;
   char name[50];
   float salary;
};
```

Here, the member **name[50]** is an array of character type and it cane be accessed using dot operator shown as follows –

```
struct employee e1;
e1.name = "Narayana"
```

Arrays of Structure

An array of structures in C can be defined as the collection of multiple structures variables where each variable contains information about different members. The array of structures in C are used to store information about multiple members of different data types. The array of structures is also known as the collection of structures. An array of structures used in grouping the records together and provides fast access.

Eg:

```
struct marks
{
        int s1;
        int s2;
        int s3;
}st [10];
```

By using the above declaration it is possible to store marks of 10 students, where each student comprised three subject marks. It is also possible to initialize an array of structure as shown in the example below.

```
struct marks  st[3] = {{70, 46, 85} {74, 80, 64,} {45, 69, 76}};
```

In Array of structures, while accessing a member it is required to specify the structure variable and its subscript along with dot operator as shown below.

```
st[0].s1= 30;
st[1].s2= 40;
st[0].s3 = 25;
```

Structures within Structures

It means nesting of structures. Nesting of structures is permitted in 'C'.

Eg:

```
struct emp
{
        int empno;
```

```
            char name [20];
            struct allowance
            {
                int da;
                int hra;
                 int pf;
            } s;
        } e1,e2,e3;
```

Now, the 'emp' structure contains another structure 'allowance' with its members. The members contained in the inner structure can be referred as -

```
        e1.s.da;
        e1.s.hra;
        e1.s.pf;
```

It is also allowed to use **tag names** to define inner structure.

Eg:
```
        struct date
        {
            int day;
            int  month;
            int year;
        };
        struct account
        {
            int act_no;
            char act_type [20];
            char cust_name [20];
            float balance;
            struct date  lastPayment;
        };
```

Structures and Pointers

C language allows to declare a pointer to structure type. This pointer is called structure pointer or pointer to structure.

Ex:
```
        struct inventory
        {
            char name [20];
            int number;
            float price;
        }product, *ptr;
```

The above statement declares the **product** as a structure variable and **ptr** as a structure pointer of type **struct inventory**.

```
        ptr = &product
```

This assignment assigns the address **product** vraible to **ptr** pointer. In this case the members can be accessed by using **ptr** as follows.

```
        ptr → name
        ptr → number
        ptr → price
```

➔ is called an arrow operator or pointer to member operator.

The following notation can also be used to access a structure member by using structure pointer **ptr**.

```
        (*ptr).name
        (*ptr).number
        (*ptr).price
```

Self –referential Structures

In a structure, if one of the members is a pointer to the parent structure type then such structure is known as **self-referential structure.**

In general terms, this can be expressed as

```
            struct  tag
            {
                member1;
                member2;
                  :
                struct tag *name;
            };
```

Example:-

```
struct list
{
        char item[40];
        struct list * next;
};
```

- Self-referential structures are very useful in applications that involve linked data structures such as linked lists and trees.
- The basic idea in a linked data structure is that each component within the structure includes a pointer indicating where the next component can be found.
- Self-referential structures are ideally suited for applications involving linked data structures. Each structure will represent a single component within the linked data structure. The self –referential pointer will indicate the location of the next component.

UNIONS

Unions are quite like the structures in C. Union is also a derived type as structure. Union can be defined in same manner as structures just the keyword used in defining union in **union** where keyword used in defining structure was **struct.**

```
union car{
  char name[50];
  int price;
};
```

Union variables can be created in a similar manner as structure variable.

```
union car{
  char name[50];
  int price;
}c1, c2, *c3;

OR;

union car{
  char name[50];
  int price;
};
-------Inside Function-----------
union car c1, c2, *c3;
```

In both cases, union variables *c1*, *c2* and union pointer variable *c3* of type **union car** is created.

Accessing members of a union

The member of unions can be accessed in similar manner as that structure. Suppose, we you want to access price for union variable *c1* in above example, it can be accessed as c1.price. If you want to access price for union pointer variable *c3*, it can be accessed as (*c3).price or as c3->price.

Difference between union and structure

1. Though unions are like structure in so many ways, the difference between them is crucial to understand. This can be demonstrated by this example:

```
#include <stdio.h>
union job {        //defining a union
   char name[32];
   float salary;
   int worker_no;
}u;
struct job1 {
   char name[32];
   float salary;
   int worker_no;
}s;
int main(){
   printf("size of union = %d",sizeof(u));
   printf("\nsize of structure = %d", sizeof(s));
```

```
  return 0;
}
```

Output
size of union = 32
size of structure = 40
There is difference in memory allocation between union and structure as suggested in above example. The amount of memory required to store a structure variable is the sum of memory size of all members.

Fig: Memory allocation in case of structure

But, the memory required to store a union variable is the memory required for largest element of an union.

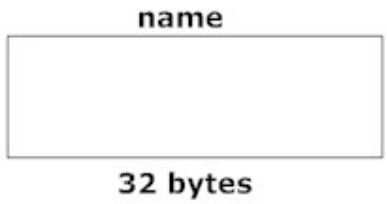


Fig: Memory allocation in case of union

2. All members of structure can be accessed at any time. But, only one member of union can be accessed at a time in case of union and other members will contain garbage value.

```
#include <stdio.h>
union job {
  char name[32];
  float salary;
  int worker_no;
}u;
int main(){
  printf("Enter name:\n");
  scanf("%s",&u.name);
  printf("Enter salary: \n");
  scanf("%f",&u.salary);
  printf("Displaying\nName :%s\n",u.name);
  printf("Salary: %.1f",u.salary);
  return 0;
}
```

Output
Enter name
Hillary
Enter salary
1234.23
Displaying
Name: f%Bary
Salary: 1234.2

Why this output is
Initially, *Hillary* will be stored in u.name and other members of union will contain garbage value. But when user enters value of salary, 1234.23 will be stored in u.salary and other members will contain garbage value. Thus in output, salary is printed accurately but, name displays some random string.

Structure Vs Union

Structure	Union
The struct keyword is used to define a structure.	The union keyword is used to define union.
When the variables are declared in a structure, the compiler allocates memory to each variable's member. The size of a structure is equal or greater to the sum of the sizes of each data member.	When the variable is declared in the union, the compiler allocates memory to the largest size variable member. The size of a union is equal to the size of its largest data member size.

Structure	Union
Each variable member occupied a unique memory space.	Variables members share the memory space of the largest size variable.
Changing the value of a member will not affect other variables members.	Changing the value of one member will also affect other variables members.
Each variable member will be assessed at a time.	Only one variable member will be assessed at a time.
We can initialize multiple variables of a structure at a time.	In union, only the first data member can be initialized.
All variable members store some value at any point in the program.	Exactly only one data member stores a value at any particular instance in the program.
The structure allows initializing multiple variable members at once.	Union allows initializing only one variable member at once.
It is used to store different data type values.	It is used for storing one at a time from different data type values.
It allows accessing and retrieving any data member at a time.	It allows accessing and retrieving any one data member at a time.

BIT FIELD

A bit field is set up with a structure declaration that labels each field and determines its width.

Declaration

The declaration of a bit-field has the form inside a structure:

```
struct
{
  type [member_name] : width ;
};
```

Below the description of variable elements of a bit field:

Elements	Description
Type	An integer type that determines how the bit-field's value is interpreted. The type may be int, signed int, unsigned int.
member_name	The name of the bit-field.
Width	The number of bits in the bit-field. The width must be less than or equal to the bit width of the specified type.

The variables defined with a predefined width are called **bit fields**. A bit field can hold more than a single bit for example if you need a variable to store a value from 0 to 7 only then you can define a bit field with a width of 3 bits as follows:

```
struct
{
  unsigned int age : 3;
} Age;
```

The above structure definition instructs C compiler that age variable is going to use only 3 bits to store the value, if you will try to use more than 3 bits then it will not allow you to do so. Let us try the following example:

```
#include <stdio.h>
#include <string.h>
struct
{
  unsigned int age : 3;
} Age;

int main( )
{
   Age.age = 4;
   printf( "Sizeof( Age ) : %d\n", sizeof(Age) );
   printf( "Age.age : %d\n", Age.age );

   Age.age = 7;
   printf( "Age.age : %d\n", Age.age );
```

```
  Age.age = 8;
  printf( "Age.age : %d\n", Age.age );

  return 0;
}
```

When the above code is compiled, it will compile with warning and when executed, it produces the following result:

```
Sizeof( Age ) : 4
Age.age : 4
Age.age : 7
Age.age : 0
```

TYPEDEF

Typedef is a keyword that is used to give a new symbolic name for the existing name in a C program. This is same like defining alias for the commands.

```
typedef   int   TypeX;     // declare a new data type TypeX
TypeX   a;              // a is of an instance of TypeX (really is an int)
typedef float  TypeY;    // declare a new type TypeY
TypeY   b,c;         // b and c are TypeY (really are floats)
```

Example program for C typedef:

```
// Structure using typedef:

#include <stdio.h>
#include <string.h>

typedef struct student
{
 int id;
 char name[20];
 float percentage;
} status;

int main()
{
 status record;
 record.id=1;
 strcpy(record.name, "Raju");
 record.percentage = 86.5;
 printf(" Id is: %d \n", record.id);
 printf(" Name is: %s \n", record.name);
 printf(" Percentage is: %f \n", record.percentage);
 return 0;
}
```

Output:

```
Id            is:          1
Name          is:       Raju
Percentage is: 86.500000
```

ENUMERATION

An enumeration is a user-defined data type consists of integral constants and each integral constant is give a name. Keyword enum is used to defined enumerated data type.

enum type_name{ value1, value2,...,valueN };

Here, *type_name* is the name of enumerated data type or tag. And *value1*, *value2*,....,*valueN* are values of type *type_name*.

By default, *value1* will be equal to 0, *value2* will be 1 and so on but, the programmer can change the default value.

```
// Changing the default value of enum elements
enum suit{
   club=0;
```

```
    diamonds=10;
    hearts=20;
    spades=3;
};
```

Declaration of enumerated variable

Above code defines the type of the data but, no any variable is created. Variable of type enum can be created as:

```
enum boolean{
    false;
    true;
};
enum boolean check;
```

Here, a variable check is declared which is of type **enum boolean**.

Example of enumerated type

```
#include <stdio.h>
enum week{ sunday, monday, tuesday, wednesday, thursday, friday, saturday};
int main(){
    enum week today;
    today=wednesday;
    printf("%d day",today+1);
    return 0;
   }
```

Output

4 day